OAVS TGT COMPUTER SCIENCE TEACHER EXAMINATION

MCQ QUESTION BANK

Topic: File Based I/O

171 Unique Exam-Oriented Multiple Choice Questions
with Answers & Concise Explanations

Concept of a File • Text Files in C • FILE Pointer • stdio.h • FILE Structure
fopen() • putc() • getc() • fclose() • fgets() • fputs() • feof() • Simple File Programs

Table of Contents

#	Chapter	Questions
1	Concept of a File	20
2	Text Files in C	20
3	Predefined FILE Pointer	15
4	Definition of FILE in stdio.h	12
5	Members of the FILE Structure	12
6	Disk I/O Functions	72
7	Simple File-Based Programs	20

Total Unique Questions: 171

Note: All questions are strictly derived from the OAVS TGT Computer Science syllabus unit “File Based I/O” and its listed sub-topics only.

CHAPTER 1

Concept of a File

Q1 | File Concept | Concept of a File *Definition*

What is a file in computer terminology?

A. A collection of related data stored on a secondary storage device

B. A temporary variable stored in RAM

C. A function used to allocate memory

D. A section of the CPU cache

Ans: A

Explanation: *A file is a named collection of related information stored permanently on a secondary storage device such as a disk.*

Q2 | File Concept | Concept of a File *Purpose*

Why is file storage used in programming?

A. To increase the speed of the CPU

B. To store data permanently even after the program terminates

C. To reduce the size of the source code

D. To compile the program faster

Ans: B

Explanation: *Files provide permanent (non-volatile) storage, so data survives after the program ends, unlike variables in RAM.*

Q3 | File Concept | Concept of a File *Characteristics*

Which of the following is a key characteristic of a file?

A. It exists only while the program is running

B. It is stored only in registers

C. It provides persistent storage of data

D. It disappears when the program executes a loop

Ans: C

Explanation: *A defining characteristic of a file is persistence -- data remains stored even after the program stops running.*

Q4 | File Concept | Concept of a File *Types*

As per the syllabus, which type of file is primarily studied?

A. Binary files
B. Executable files
C. Compressed files
D. Text files

Ans: D

Explanation: *The syllabus restricts coverage to text files; binary files are outside its scope.*

Q5 | File Concept | Concept of a File *Advantages*

What is one major advantage of storing data in files?

A. Data can be reused without re-entering it every time the program runs
B. Files always execute faster than arrays
C. Files automatically debug the program
D. Files replace the need for variables

Ans: A

Explanation: *A key advantage of files is reusability -- stored data can be accessed again without retyping it.*

Q6 | File Concept | Concept of a File *Applications*

Which of the following is a common application of files in C programs?

A. Increasing the compiler speed
B. Storing student records for later retrieval
C. Allocating stack memory
D. Managing CPU registers

Ans: B

Explanation: *Files are commonly used to store structured data, such as records, that can be retrieved in future program runs.*

Q7 | File Concept | Concept of a File *Fill-in-the-Blank*

A file must generally be associated with a _____ before it can be used in a C program.

A. loop counter
B. array index
C. FILE pointer
D. register variable

Ans: C

Explanation: *Before performing operations, a file must be linked to a FILE pointer obtained through fopen().*

Q8 | File Concept | Concept of a File *Statement-based*

Which statement about files is TRUE?

A. Files are erased automatically when the program ends
B. Files can only store integer values
C. Files are part of the CPU
D. Files allow data to persist beyond the lifetime of the program

Ans: D

Explanation: *Persistence of data beyond program execution is the fundamental purpose of files.*

Q9 | File Concept | Concept of a File *Basic*

What basic operation is essential to using any file in C?

A. Opening the file before reading or writing
B. Compiling the file separately
C. Declaring the file as an array
D. Registering the file with the operating system manually each time

Ans: A

Explanation: *A file must be opened with a function like fopen() before any read/write operation can be performed.*

Q10 | File Concept | Concept of a File *Statement-based*

Which of these is NOT a characteristic of a file?

A. It stores data on secondary storage

B. It is deleted automatically at the end of every function call

C. It has a name for identification

D. It can be opened, read, written and closed

Ans: B

Explanation: *Files persist independently of function calls; they are not automatically deleted when a function ends.*

Q11 | File Concept | Concept of a File *Purpose*

The primary purpose of file storage in a program is to:

A. Speed up arithmetic operations

B. Increase variable scope

C. Preserve data permanently for future use

D. Reduce the number of functions

Ans: C

Explanation: *File storage exists mainly to make data available even after the program has finished executing.*

Q12 | File Concept | Concept of a File *Definition*

Which of the following best defines 'file handling' in C?

A. The process of debugging syntax errors

B. The process of compiling multiple files together

C. The process of allocating memory dynamically

D. The process of creating, reading, writing, and closing files using C functions

Ans: D

Explanation: *File handling refers to the set of operations (open, read, write, close) performed on files using C's standard library functions.*

Q13 | File Concept | Concept of a File *Concept-based*

Data stored in a file remains available:

A. Even after the program that created it has terminated

B. Only during program execution

C. Only inside a single function

D. Only until the next line of code executes

Ans: A

Explanation: *Unlike variables, file data is non-volatile and persists after program termination.*

Q14 | File Concept | Concept of a File *Applications*

Which of the following is an application area where file concepts are essential?

A. Improving loop iteration speed

B. Maintaining logs and records over multiple program runs

C. Reducing variable declarations

D. Managing arithmetic overflow

Ans: B

Explanation: *Logging and record-keeping across multiple executions are classic applications requiring file storage.*

Q15 | File Concept | Concept of a File *Comparison-based*

A file, unlike a variable, is stored on:

A. The CPU register

B. Primary (volatile) memory only

C. Secondary (non-volatile) storage

D. Cache memory only

Ans: C

Explanation: *Files reside on secondary storage devices, making them non-volatile compared to variables in RAM.*

Q16 | File Concept | Concept of a File *Basic*

Which is a basic requirement before writing data to a file?

A. The file must be converted to binary first

B. The program must be run twice

C. The variable must be declared as global

D. The file must be opened in a mode that permits writing

Ans: D

Explanation: *Writing to a file requires opening it in an appropriate mode such as "w" or "a".*

Q17 | File Concept | Concept of a File *Comparison-based*

What happens to unsaved data in a variable when the program terminates, compared to data in a file?

A. Variable data is lost, but file data remains stored

B. Both are lost equally

C. Both remain stored permanently

D. Variable data is stored, file data is lost

Ans: A

Explanation: *Variables exist only during execution (volatile), while files retain data permanently (non-volatile).*

Q18 | File Concept | Concept of a File *Advantages*

Which of these is an advantage of using files over keeping data only in variables?

A. Files execute arithmetic operations faster

B. Large volumes of data can be stored beyond the limits of RAM

C. Files remove the need for a compiler

D. Files reduce the number of keywords in C

Ans: B

Explanation: *Files can hold far larger amounts of data than can practically be kept in memory-resident variables.*

Q19 | File Concept | Concept of a File *Concept-based*

The concept of a 'file' in C programming is most closely related to:

A. A block of executable machine code only

B. A single integer variable

C. A stream of data linked to a named entity on storage

D. A CPU instruction cycle

Ans: C

Explanation: *In C, a file is treated as a stream of data associated with a named object on secondary storage.*

Q20 | File Concept | Concept of a File *Fill-in-the-Blank*

A file provides a means of _____ data between a program and secondary storage.

A. compiling

B. encrypting

C. registering

D. transferring

Ans: D

Explanation: *A file acts as a channel for transferring data between the program and the storage device.*

CHAPTER 2
Text Files in C

Q21 | Text Files | Text Files in C *Definition*

What is a text file?

A. A file that stores data as a sequence of readable ASCII characters

B. A file that stores data only in binary machine code

C. A file that stores only images

D. A file that can only be opened by a compiler

Ans: A

Explanation: *Text files store human-readable characters (typically ASCII), organized into lines.*

Q22 | Text Files | Text Files in C *Characteristics*

Which of the following best describes a characteristic of text files?

A. Data is stored exactly as it appears in memory in binary form

B. Data is stored line by line as readable characters

C. Data cannot be edited using a text editor

D. Data occupies less space than any other file type in all cases

Ans: B

Explanation: *Text files organize data into lines of readable characters, which can be opened directly in a text editor.*

Q23 | Text Files | Text Files in C *Syntax-based*

A text file in C is typically organized into lines using which special character?

A. Null character ('\0') only

B. Tab character ('\t')

C. Newline character ('\n')

D. Percent character ('%')

Ans: C

Explanation: *Lines in a text file are conventionally separated using the newline character \n.*

Q24 | Text Files | Text Files in C *Syntax-based*

Which mode is commonly used to create a new text file for writing in C?

A. "r"

B. "x"

C. "e"

D. "w"

Ans: D

Explanation: *The "w" mode creates a new text file for writing, or truncates it if it already exists.*

Q25 | Text Files | Text Files in C *Syntax-based*

To read data from an existing text file, which mode should be used with fopen()?

A. "r"
B. "w"
C. "a"
D. "n"

Ans: A

Explanation: *The "r" mode opens an existing text file for reading only.*

Q26 | Text Files | Text Files in C *Advantages*

Which of the following is an advantage of text files?

A. They always occupy the least possible disk space
B. They are human-readable and easy to edit with standard text editors
C. They cannot be corrupted
D. They execute faster than all other data structures

Ans: B

Explanation: *A key advantage of text files is that their contents can be viewed and edited directly using ordinary text editors.*

Q27 | Text Files | Text Files in C *Limitations*

Which of the following is a limitation of text files?

A. They cannot store alphabetic characters
B. They cannot be opened by any editor
C. Storing numeric data as text can require more space than storing it in binary form
D. They must always be encrypted

Ans: C

Explanation: *Since numbers are stored as character sequences, text files can need more space than an equivalent binary representation.*

Q28 | Text Files | Text Files in C *Representation*

A text file storing the number 12345 will store it as:

A. A 4-byte binary integer

B. A single byte value

C. A floating-point binary value

D. The characters '1','2','3','4','5' in sequence

Ans: D

Explanation: *In a text file, numeric data is represented as a sequence of character digits, not as raw binary values.*

Q29 | Text Files | Text Files in C *Function-based*

Which function is commonly used to create and write to a text file character by character?

A. putc()

B. fread()

C. fseek()

D. remove()

Ans: A

Explanation: *putc() writes a single character to an open text file, useful for creating file content character by character.*

Q30 | Text Files | Text Files in C *Applications*

What is a common application of text files?

A. Storing raw executable machine instructions

B. Storing configuration settings or plain textual records

C. Managing CPU cache

D. Allocating heap memory

Ans: B

Explanation: *Text files are widely used to store human-readable data such as configuration values or simple records.*

Q31 | Text Files | Text Files in C *Concept-based*

Which of the following correctly represents reading a text file line by line?

A. Using fopen() repeatedly for every line

B. Using putc() to read data

C. Using fgets() in a loop until end-of-file is reached

D. Using fclose() before reading each line

Ans: C

Explanation: *fgets() reads one line at a time from a text file and is typically used inside a loop until EOF.*

Q32 | Text Files | Text Files in C *Comparison-based*

A text file differs from a binary file mainly because:

A. A text file cannot be opened by any program

B. A binary file always contains fewer bytes

C. A text file is stored only in RAM

D. A text file stores data as readable characters, while a binary file stores raw byte data

Ans: D

Explanation: *The essential difference lies in representation -- text files use readable characters, binary files store raw bytes.*

Q33 | Text Files | Text Files in C *Creation*

Which of these best describes the creation of a text file in C?

A. Opening a file in write mode using fopen() and writing character or string data to it

B. Declaring an array of integers

C. Compiling the C program twice

D. Allocating memory using malloc() only

Ans: A

Explanation: *Text file creation involves opening the file in a write mode and using write functions such as fputs() or putc().*

Q34 | Text Files | Text Files in C *Syntax-based*

Appending new data to the end of an existing text file requires opening it in mode:

A. "r"
B. "a"
C. "w"
D. "t"

Ans: B

Explanation: *The "a" (append) mode positions the file pointer at the end of the file so new data is added without erasing existing content.*

Q35 | Text Files | Text Files in C *Limitations*

Which statement about text file limitations is correct?

A. Text files cannot store any data
B. Text files can only be read once
C. Text files can be less efficient for storing large amounts of structured numeric data
D. Text files must always be smaller than binary files

Ans: C

Explanation: *Because numeric values are stored as characters, text files can be less space-efficient than binary files for numeric-heavy data.*

Q36 | Text Files | Text Files in C *Function-based*

When writing strings to a text file, which function is most appropriate?

A. getc()
B. feof()
C. fclose() alone
D. fputs()

Ans: D

Explanation: *fputs() is designed to write a string of characters to a text file.*

Q37 | Text Files | Text Files in C *Statement-based*

Which of the following is TRUE about reading and writing text files in C?

A. Both operations require the file to first be opened with an appropriate mode

B. Reading requires opening the file, but writing does not

C. Neither reading nor writing requires opening the file

D. Only writing requires closing the file afterward

Ans: A

Explanation: *Both reading and writing operations on a text file require the file to be opened in a suitable mode beforehand.*

Q38 | Text Files | Text Files in C *Applications*

A text file is best suited for which of the following applications?

A. Storing compiled machine code for direct execution

B. Storing plain textual logs that a human can review directly

C. Managing hardware interrupts

D. Allocating stack frames

Ans: B

Explanation: *Text files are ideal for human-readable logs, notes, and simple records.*

Q39 | Text Files | Text Files in C *Representation*

Which representation is used internally when a text file is processed on most systems?

A. A sequence of raw memory addresses

B. A single continuous integer array

C. A sequence of characters, often with newline characters separating lines

D. A set of CPU instructions

Ans: C

Explanation: *Text files are represented as sequences of characters, organized into lines using newline characters.*

Q40 | Text Files | Text Files in C *Application-oriented*

What advantage does a text file have when debugging a C program's output?

A. The output automatically fixes syntax errors

B. The output is compiled into an executable

C. The output increases program execution speed

D. The output can be directly viewed and understood without a special program

Ans: D

Explanation: *Because text files are human-readable, they make it easy to inspect a program's output while debugging.*

CHAPTER 3

Predefined FILE Pointer

Q41 | FILE Pointer | Predefined FILE Pointer *Definition*

What is FILE in C?

A. A predefined structure type used to store information about a file

B. A keyword used to declare integer variables

C. A built-in function to open files

D. A loop control statement

Ans: A

Explanation: *FILE is a structure type defined in stdio.h that holds information required to manage an open file.*

Q42 | FILE Pointer | Predefined FILE Pointer *Purpose*

The purpose of the FILE pointer is to:

A. Store the source code of the program

B. Keep track of the current position and status of an open file

C. Allocate memory for arrays

D. Control loop iterations

Ans: B

Explanation: *A FILE pointer references a FILE structure that tracks the current read/write position and status of the file.*

Q43 | FILE Pointer | Predefined FILE Pointer *Syntax-based*

Which of the following is the correct declaration syntax for a FILE pointer?

A. file fp;
B. FILE fp[];
C. FILE *fp;
D. int *FILE;

Ans: C

Explanation: *A FILE pointer is declared using the syntax FILE *fp, where fp is the pointer variable name.*

Q44 | FILE Pointer | Predefined FILE Pointer *Function-based*

A FILE pointer is returned by which function when a file is successfully opened?

A. getc()
B. feof()
C. putc()
D. fopen()

Ans: D

Explanation: *fopen() returns a pointer to a FILE structure representing the opened file, or NULL on failure.*

Q45 | FILE Pointer | Predefined FILE Pointer *Concept-based*

Which of the following is a standard predefined file pointer available in C without explicit opening?

A. stdin
B. myfile
C. datafile
D. logfile

Ans: A

Explanation: *stdin, stdout, and stderr are standard file pointers automatically available for console input/output.*

Q46 | FILE Pointer | Predefined FILE Pointer *Concept-based*

What is the relationship between a FILE pointer and file handling functions like fgets() or fputs()?

A. These functions never use a FILE pointer

B. These functions require a FILE pointer as an argument to identify which file to operate on

C. The FILE pointer is only used with fopen()

D. These functions replace the need for a FILE pointer

Ans: B

Explanation: *File handling functions use the FILE pointer to know which specific open file stream to read from or write to.*

Q47 | FILE Pointer | Predefined FILE Pointer *Statement-based*

If fopen() fails to open a file, what value does the FILE pointer typically hold?

A. Zero-length string

B. -1

C. NULL

D. An uninitialized garbage integer

Ans: C

Explanation: *fopen() returns NULL if the file could not be opened, which should always be checked before use.*

Q48 | FILE Pointer | Predefined FILE Pointer *Concept-based*

Why is a FILE pointer necessary before using functions like getc() or putc()?

A. Because these functions cannot execute without a loop

B. Because C requires all pointers to be of type FILE

C. Because it improves compilation speed

D. Because these functions need to know which specific file stream to read from or write to

Ans: D

Explanation: *The FILE pointer identifies the exact stream so getc()/putc() know which file to act upon.*

Q49 | FILE Pointer | Predefined FILE Pointer *Concept-based*

What does the FILE pointer essentially represent?

A. A reference to the internal structure managing an open file's data and state

B. The actual data stored inside the file

C. The name of the file only

D. The size of the file in bytes

Ans: A

Explanation: *The FILE pointer references the internal FILE structure, not the raw file content itself.*

Q50 | FILE Pointer | Predefined FILE Pointer *Basic*

Before performing any read or write operation, a program must first:

A. Declare the FILE pointer as static

B. Assign the return value of fopen() to a FILE pointer

C. Convert the FILE pointer to an integer

D. Call feof() first

Ans: B

Explanation: *A valid FILE pointer obtained from fopen() is required before any file I/O operation can proceed.*

Q51 | FILE Pointer | Predefined FILE Pointer *Code-snippet*

Which of these correctly uses a FILE pointer to open a file for reading?

A. FILE fp = open("data.txt");

B. int fp = fopen("data.txt", "r");

C. FILE *fp = fopen("data.txt", "r");

D. FILE *fp = read("data.txt");

Ans: C

Explanation: *fopen() returns a FILE pointer, which should be assigned to a variable declared as FILE *.*

Q52 | FILE Pointer | Predefined FILE Pointer *Purpose*

What is the primary use of a FILE pointer in file handling functions?

A. To store the size of RAM available

B. To specify the CPU clock speed

C. To count loop iterations

D. To act as a handle that identifies the specific open file being accessed

Ans: D

Explanation: *The FILE pointer serves as a handle so functions know exactly which open file stream to operate on.*

Q53 | FILE Pointer | Predefined FILE Pointer *Statement-based*

Which statement about the FILE pointer is TRUE?

A. It must be checked for NULL after calling fopen() to confirm the file opened successfully

B. It is guaranteed to never be NULL

C. It automatically closes the file when declared

D. It cannot be used with fclose()

Ans: A

Explanation: *Checking for NULL after fopen() ensures the file was opened successfully before further operations.*

Q54 | FILE Pointer | Predefined FILE Pointer *Concept-based*

A single C program can declare:

A. Only one FILE pointer per program

B. Multiple FILE pointers to handle multiple files simultaneously

C. No more than two FILE pointers

D. FILE pointers only inside main()

Ans: B

Explanation: *A program can declare as many FILE pointers as needed to work with multiple files at once.*

Q55 | FILE Pointer | Predefined FILE Pointer *Definition*

What does 'predefined' mean in the context of the FILE pointer type?

A. The FILE pointer's value is fixed and cannot change

B. The FILE pointer is defined only inside main()

C. The FILE type is already defined in the standard library and does not need to be defined by the programmer

D. The FILE pointer is predefined as NULL always

Ans: C

Explanation: *FILE is a type predefined in stdio.h, so programmers can use it directly without defining it themselves.*

CHAPTER 4

Definition of FILE in stdio.h

Q56 | stdio.h | Definition of FILE in stdio.h *Basic*

In which header file is the FILE type defined?

A. stdlib.h

B. string.h

C. conio.h

D. stdio.h

Ans: D

Explanation: *The FILE type, along with standard I/O functions, is defined in the stdio.h header file.*

Q57 | stdio.h | Definition of FILE in stdio.h *Purpose*

What is the main purpose of including stdio.h in a C program that performs file operations?

A. To gain access to the FILE type and standard input/output functions

B. To enable dynamic memory allocation

C. To perform mathematical calculations

D. To enable string comparison functions

Ans: A

Explanation: *stdio.h provides the declarations for FILE and functions such as fopen(), fclose(), fgets(), etc.*

Q58 | stdio.h | Definition of FILE in stdio.h *Function-based*

Which of the following functions is declared in stdio.h?

A. malloc()
B. fopen()
C. strcpy()
D. sqrt()

Ans: B

Explanation: *fopen(), along with other file handling functions, is declared in the stdio.h header.*

Q59 | stdio.h | Definition of FILE in stdio.h *Importance*

Why is stdio.h considered important for file-based programs in C?

A. It manages the CPU scheduler
B. It optimizes loop execution
C. It provides the standard functions and types required for file input/output operations
D. It provides graphics rendering support

Ans: C

Explanation: *stdio.h supplies the essential building blocks (FILE type, fopen, fclose, etc.) needed for file handling.*

Q60 | stdio.h | Definition of FILE in stdio.h *Definition*

FILE is best described as a:

A. Keyword reserved only for arrays
B. Preprocessor macro for comments
C. Function that reads integers
D. Type defined via typedef in stdio.h representing file stream information

Ans: D

Explanation: *FILE is typically implemented as a typedef'd structure in stdio.h that stores stream information.*

Q61 | stdio.h | Definition of FILE in stdio.h *Concept-based*

Without including stdio.h, which of the following would generally fail to compile correctly?

A. Declaring a variable of type FILE *

B. Declaring an integer variable

C. Declaring a for loop

D. Declaring a character array

Ans: A

Explanation: *Since FILE is defined in stdio.h, omitting this header would prevent the compiler from recognizing the FILE type.*

Q62 | stdio.h | Definition of FILE in stdio.h *Standard Library*

Which of these is a standard library support feature provided by stdio.h?

A. Support for network socket programming

B. Functions for formatted and unformatted input/output on files and console

C. Support for graphical user interfaces

D. Support for multithreading

Ans: B

Explanation: *stdio.h focuses on standard input/output operations, including file and console I/O.*

Q63 | stdio.h | Definition of FILE in stdio.h *Importance*

The importance of stdio.h in file handling lies in its provision of:

A. Only mathematical constants

B. Only error codes for the operating system

C. Both the FILE type and the function prototypes needed to manipulate files

D. Only screen color codes

Ans: C

Explanation: *stdio.h bundles together the FILE type definition and prototypes of file handling functions.*

Q64 | stdio.h | Definition of FILE in stdio.h *Statement-based*

Which of the following statements is TRUE about the FILE type in stdio.h?

A. It requires the programmer to manage raw disk sectors manually

B. It is only used for binary files

C. It eliminates the need for fopen()

D. It abstracts the low-level details of file management from the programmer

Ans: D

Explanation: *The FILE type hides internal implementation details, letting programmers work with files through simple function calls.*

Q65 | stdio.h | Definition of FILE in stdio.h *Concept-based*

Standard file pointers like stdin and stdout are declared as being of type:

A. FILE *

B. int

C. char *

D. void

Ans: A

Explanation: *stdin, stdout, and stderr are all predeclared pointers of type FILE * in stdio.h.*

Q66 | stdio.h | Definition of FILE in stdio.h *Uses*

What role does stdio.h play with respect to portability of file handling code?

A. It makes the code run only on one specific operating system

B. It provides a standardized interface so file operations work consistently across different systems

C. It removes the need for a compiler

D. It converts C code into machine code directly

Ans: B

Explanation: *stdio.h offers a standard, portable interface for file operations across different platforms.*

Q67 | stdio.h | Definition of FILE in stdio.h *Applications*

Which is an example use of stdio.h beyond file handling?

A. Managing dynamic memory allocation

B. Performing string manipulation like strcmp()

C. Performing console input and output using functions like printf() and scanf()

D. Handling mathematical operations like pow()

Ans: C

Explanation: *stdio.h also supports standard input/output on the console through functions such as printf() and scanf().*

CHAPTER 5
Members of the FILE Structure

Q68 | FILE Structure | Members of the FILE Structure *Members*

The FILE structure generally maintains information about:

A. The color of the terminal window

B. The number of variables declared in the program

C. The CPU's clock speed

D. The current position, status, and buffering details of an open file

Ans: D

Explanation: *The FILE structure holds internal details such as current position and buffer/status information for the open file.*

Q69 | FILE Structure | Members of the FILE Structure *Purpose*

What is the general purpose of the members within the FILE structure?

A. To help the library functions manage and track the state of the open file internally

B. To store the entire source code of the program

C. To act as a substitute for the file pointer itself

D. To increase the readability of the printed file content

Ans: A

Explanation: *FILE structure members support internal bookkeeping so that file handling functions can operate correctly.*

Q70 | FILE Structure | Members of the FILE Structure *Role*

Which of the following is a conceptual role of the FILE structure's members?

A. Compiling the program into an executable

B. Tracking whether the end of the file has been reached

C. Declaring global variables

D. Managing the operating system's process table

Ans: B

Explanation: *Among its roles, the FILE structure tracks status flags, including whether the end-of-file condition has occurred.*

Q71 | FILE Structure | Members of the FILE Structure *Concept-based*

The FILE structure's members work together with which type of variable to manage a file?

A. An integer array

B. A string constant

C. The FILE pointer that references the structure

D. A macro definition

Ans: C

Explanation: *The FILE pointer references the FILE structure whose members are used internally during file operations.*

Q72 | FILE Structure | Members of the FILE Structure *Practical Understanding*

In practical use, a C programmer typically:

A. Directly edits each member of the FILE structure manually

B. Redefines the FILE structure every time a file is opened

C. Removes members from the FILE structure before use

D. Does not access the FILE structure's members directly, but uses library functions instead

Ans: D

Explanation: *Programmers generally interact with files through standard functions rather than manipulating FILE structure members directly.*

Q73 | FILE Structure | Members of the FILE Structure *Working Concepts*

Why is a basic understanding of FILE structure members useful, even though direct access is uncommon?

A. It helps in understanding how functions like feof() and fclose() work internally

B. It is required to write any C program

C. It replaces the need to learn fopen()

D. It allows bypassing the compiler

Ans: A

Explanation: *Knowing the conceptual role of FILE structure members builds a clearer mental model of how file functions operate.*

Q74 | FILE Structure | Members of the FILE Structure *Working Concepts*

Which of the following best matches the working concept of the FILE structure?

A. It acts as the actual physical file on disk

B. It acts as a container that stores metadata the library uses to manage the file stream

C. It acts as a compiler directive

D. It acts as a printing device driver

Ans: B

Explanation: *The FILE structure is metadata used by the standard library to manage the stream, not the physical file itself.*

Q75 | FILE Structure | Members of the FILE Structure *Role*

The FILE structure's members assist which of the following operations?

A. Sorting arrays

B. Allocating heap memory for structures unrelated to files

C. Buffered reading and writing of file data

D. Managing recursive function calls

Ans: C

Explanation: *Members of the FILE structure support buffering, which underlies efficient reading and writing operations.*

Q76 | FILE Structure | Members of the FILE Structure *Meaning*

According to the syllabus scope, which level of understanding of FILE structure members is expected?

A. A detailed, compiler-specific memory layout of every member

B. Complete knowledge of assembly-level implementation

C. Ability to redesign the FILE structure

D. A basic, conceptual understanding of their purpose, not implementation-specific internal details

Ans: D

Explanation: *The syllabus restricts coverage to basic conceptual understanding, excluding compiler-dependent internal details.*

Q77 | FILE Structure | Members of the FILE Structure *Purpose*

The FILE structure plays a role in file handling primarily by:

A. Providing the internal state that functions use to correctly perform read/write operations

B. Directly displaying file content on the screen

C. Replacing the need for a FILE pointer entirely

D. Formatting the source code

Ans: A

Explanation: *The FILE structure's internal state enables file functions to correctly track and perform operations on a stream.*

Q78 | FILE Structure | Members of the FILE Structure *Working Concepts*

Which of the following is a working concept associated with the FILE structure?

A. It stores unrelated user login credentials

B. It helps the system know the current read/write position within a file

C. It is used to compile other C programs

D. It manages RAM allocation for arrays

Ans: B

Explanation: *One key working concept is that the structure keeps track of the current position for the next read/write operation.*

Q79 | FILE Structure | Members of the FILE Structure *Practical Understanding*

The practical understanding expected of the FILE structure, per the syllabus, focuses on:

A. Memory addresses of each member on every system

B. Rewriting the structure for custom hardware

C. How its members support the overall functioning of file handling operations

D. Bypassing fopen() using the structure directly

Ans: C

Explanation: *The expected focus is a general grasp of how the structure's members support file handling, not deep technical internals.*

CHAPTER 6
Disk I/O Functions

Function Focus: fopen()

Q80 | fopen() | Disk I/O Functions *Purpose*

What is the purpose of the fopen() function?

A. To close an already open file

B. To delete a file from storage

C. To read a single character from a file

D. To open a file and associate it with a FILE pointer

Ans: D

Explanation: *fopen() opens a specified file and returns a FILE pointer used for further operations.*

Q81 | fopen() | Disk I/O Functions *Syntax*

What is the correct syntax for fopen()?

A. FILE *fopen(const char *filename, const char *mode);

B. void fopen(char filename);

C. int fopen(FILE *fp);

D. FILE fopen(int mode);

Ans: A

Explanation: *fopen() takes a filename and a mode string, and returns a FILE pointer.*

Q82 | fopen() | Disk I/O Functions *Parameters*

How many parameters does fopen() require?

A. One: only the filename

B. Two: the filename and the mode

C. Three: filename, mode, and size

D. Zero: it requires no parameters

Ans: B

Explanation: *fopen() requires two parameters -- the name of the file and the mode in which to open it.*

Q83 | fopen() | Disk I/O Functions *Return Value*

What does fopen() return if the file cannot be opened?

A. 0

B. -1

C. NULL

D. EOF

Ans: C

Explanation: *fopen() returns NULL when the file cannot be opened, such as when it does not exist in read mode.*

Q84 | fopen() | Disk I/O Functions *File Opening Modes*

Which mode opens a text file for reading only?

A. "w"
B. "a"
C. "rw"
D. "r"

Ans: D

Explanation: *The "r" mode opens an existing file strictly for reading.*

Q85 | fopen() | Disk I/O Functions *File Opening Modes*

Which mode should be used to create a new file for writing, overwriting if it exists?

A. "w"
B. "r"
C. "a"
D. "x"

Ans: A

Explanation: *The "w" mode creates a new file or truncates an existing file for writing.*

Q86 | fopen() | Disk I/O Functions *File Opening Modes*

Which mode opens a file for appending data at the end without deleting existing content?

A. "r"
B. "a"
C. "w"
D. "n"

Ans: B

Explanation: *The "a" mode opens a file for writing, positioning the file pointer at the end so new data is appended.*

Q87 | fopen() | Disk I/O Functions *Return Value*

What does fopen() typically return upon success?

A. The number of bytes in the file
B. The integer 1
C. A valid FILE pointer to the opened file
D. A string containing the file's name

Ans: C

Explanation: *On success, fopen() returns a valid FILE pointer that can be used with other file functions.*

Q88 | fopen() | Disk I/O Functions *Code-snippet*

In FILE *fp = fopen("data.txt", "r");, what does "data.txt" represent?

A. The mode of the file
B. The FILE pointer variable
C. A comment in the code
D. The name of the file to be opened

Ans: D

Explanation: *"data.txt" is the filename parameter passed to fopen() specifying which file to open.*

Q89 | fopen() | Disk I/O Functions *Applications*

Which of the following is an application of fopen()?

A. Initiating access to a file before any read or write operation
B. Terminating the program
C. Declaring a global variable
D. Sorting an array of integers

Ans: A

Explanation: *fopen() is used at the start of file operations to establish access to the desired file.*

Q90 | fopen() | Disk I/O Functions *Common Mistakes*

What is a common mistake when using fopen()?

A. Providing a filename as a string

B. Not checking whether the returned FILE pointer is NULL before using it

C. Assigning the return value to a FILE pointer

D. Passing a mode string as the second argument

Ans: B

Explanation: *Failing to check for a NULL return value can lead to undefined behavior if the file failed to open.*

Q91 | fopen() | Disk I/O Functions *File Opening Modes*

Which mode combination would open a text file for both reading and writing without deleting existing content?

A. "w"

B. "a"

C. "r+"

D. "r"

Ans: C

Explanation: *The "r+" mode opens an existing file for both reading and writing, preserving its existing content.*

Function Focus: putc()

Q92 | putc() | Disk I/O Functions *Purpose*

What is the purpose of putc()?

A. To read a single character from a file

B. To close a file

C. To open a file for writing

D. To write a single character to a file

Ans: D

Explanation: *putc() writes one character at a time to the specified output stream.*

Q93 | putc() | Disk I/O Functions *Syntax*

What is the correct syntax for putc()?

A. int putc(int char, FILE *stream);
B. void putc(FILE *stream);
C. char putc(int char);
D. int putc(char *stream);

Ans: A

Explanation: *putc() takes a character (as an int) and a FILE pointer, writing that character to the stream.*

Q94 | putc() | Disk I/O Functions *Parameters*

How many parameters does putc() take?

A. One: only the character
B. Two: the character to write and the file pointer
C. Three: character, filename, and mode
D. Zero

Ans: B

Explanation: *putc() requires the character to be written and the FILE pointer identifying the destination stream.*

Q95 | putc() | Disk I/O Functions *Return Value*

What does putc() return on success?

A. NULL
B. The size of the file
C. The character that was written
D. The number 0 always

Ans: C

Explanation: *On success, putc() returns the character written; on failure, it returns EOF.*

Q96 | putc() | Disk I/O Functions *Code-snippet*

Which of these correctly uses putc() to write the character 'A' to a file pointer fp?

A. putc(fp, 'A');
B. putc("A", fp);
C. putc(fp);
D. putc('A', fp);

Ans: D

Explanation: *The character comes first, followed by the FILE pointer, as in putc('A', fp).*

Q97 | putc() | Disk I/O Functions *Return Value*

What does putc() return if it fails to write the character?

A. EOF
B. NULL
C. 0
D. -2

Ans: A

Explanation: *putc() returns EOF if an error occurs during the write operation.*

Q98 | putc() | Disk I/O Functions *File Opening Modes*

Which mode must the file be opened in before putc() can be used successfully?

A. Only "r" mode
B. A writable mode such as "w", "a", or "r+"
C. No mode is required
D. Only binary read mode

Ans: B

Explanation: *The file must be opened in a mode that permits writing for putc() to succeed.*

Q99 | putc() | Disk I/O Functions *Applications*

What is a common application of putc()?

A. Reading a formatted string from a file

B. Closing multiple files simultaneously

C. Writing text to a file one character at a time inside a loop

D. Renaming a file

Ans: C

Explanation: *putc() is often used within a loop to write character data sequentially to a file.*

Q100 | putc() | Disk I/O Functions *Common Mistakes*

Which of the following is a common mistake when using putc()?

A. Passing the character as the first argument

B. Passing the FILE pointer as the second argument

C. Using putc() inside a for loop

D. Forgetting to open the file in a writable mode before calling putc()

Ans: D

Explanation: *Attempting to write with putc() to a file opened only for reading will fail.*

Q101 | putc() | Disk I/O Functions *Comparison-based*

putc() is functionally similar to which other C function (though implemented as a macro in some libraries)?

A. fputc()

B. fgets()

C. fscanf()

D. fclose()

Ans: A

Explanation: *putc() and fputc() perform the same task of writing a single character, though putc() may be implemented as a macro.*

Function Focus: getc()

Q102 | getc() | Disk I/O Functions *Purpose*

What is the purpose of getc()?

A. To write a single character to a file
B. To read a single character from a file
C. To open a file
D. To close a file

Ans: B

Explanation: *getc() reads and returns the next character from the specified input stream.*

Q103 | getc() | Disk I/O Functions *Syntax*

What is the correct syntax for getc()?

A. char getc(int stream);
B. void getc(FILE *stream);
C. int getc(FILE *stream);
D. int getc(char *stream);

Ans: C

Explanation: *getc() takes a FILE pointer and returns the character read as an int.*

Q104 | getc() | Disk I/O Functions *Return Value*

What does getc() return when the end of the file is reached?

A. NULL
B. 0
C. -2
D. EOF

Ans: D

Explanation: *getc() returns the special value EOF to indicate that the end of the file has been reached.*

Q105 | getc() | Disk I/O Functions *File Opening Modes*

Which mode must a file be opened in for getc() to work correctly?

A. A readable mode such as "r" or "r+"
B. Only "w" mode
C. Only "a" mode
D. No mode is required

Ans: A

Explanation: *getc() requires the file to be opened in a mode that permits reading.*

Q106 | getc() | Disk I/O Functions *Return Value*

What type of value does getc() return for a successfully read character?

A. A string containing the character
B. An integer representing the character read
C. A pointer to the character
D. A boolean value

Ans: B

Explanation: *getc() returns the character as an int, even though it represents a single character.*

Q107 | getc() | Disk I/O Functions *Code-snippet*

Which of the following demonstrates correct usage of getc() to read a character into variable ch from fp?

A. ch = getc(ch, fp);
B. getc(fp) = ch;
C. ch = getc(fp);
D. ch = getc();

Ans: C

Explanation: *getc() is called with the FILE pointer as its only argument, and its return value is assigned to a variable.*

Q108 | getc() | Disk I/O Functions *Applications*

What is a common application of getc()?

A. Writing formatted output to the console

B. Renaming a file

C. Allocating dynamic memory

D. Reading a file character by character inside a loop until EOF is reached

Ans: D

Explanation: *getc() is typically used in a loop to process a file one character at a time until reaching EOF.*

Q109 | getc() | Disk I/O Functions *Concept-based*

Why is the return type of getc() declared as int rather than char?

A. So it can represent all possible character values as well as the special EOF value

B. Because char cannot store ASCII values

C. Because getc() actually reads integers, not characters

D. Because int uses less memory than char

Ans: A

Explanation: *Using int allows getc() to return EOF, a value outside the normal range of char, in addition to all character values.*

Q110 | getc() | Disk I/O Functions *Common Mistakes*

Which of the following is a common mistake when using getc()?

A. Using getc() with a valid FILE pointer

B. Storing the return value of getc() in a variable of type char, which can cause EOF detection to fail on some systems

C. Using getc() inside a while loop

D. Comparing the return value of getc() to EOF

Ans: B

Explanation: *Storing the result in a char variable can cause issues detecting EOF correctly, so an int variable should be used.*

Q111 | getc() | Disk I/O Functions *Comparison-based*

getc() is closely related to which other standard function that performs the same basic task?

A. fputs()
B. fopen()
C. fgetc()
D. fclose()

Ans: C

Explanation: *getc() and fgetc() both read a single character, though getc() may be implemented as a macro.*

Function Focus: fclose()

Q112 | fclose() | Disk I/O Functions *Purpose*

What is the purpose of fclose()?

A. To open a new file
B. To read data from a file
C. To rename a file
D. To close an open file and release its associated resources

Ans: D

Explanation: *fclose() closes a previously opened file, flushing any buffered data and freeing resources.*

Q113 | fclose() | Disk I/O Functions *Syntax*

What is the correct syntax for fclose()?

A. int fclose(FILE *stream);
B. void fclose(char *stream);
C. FILE fclose(int stream);
D. int fclose(char *filename);

Ans: A

Explanation: *fclose() takes a FILE pointer as its parameter and returns an integer status code.*

Q114 | fclose() | Disk I/O Functions *Return Value*

What does fclose() return on success?

A. EOF
B. 0
C. NULL
D. 1

Ans: B

Explanation: *fclose() returns 0 if the file was closed successfully.*

Q115 | fclose() | Disk I/O Functions *Return Value*

What does fclose() return if an error occurs while closing the file?

A. 0
B. NULL
C. EOF
D. -2

Ans: C

Explanation: *fclose() returns EOF to indicate that an error occurred during the closing process.*

Q116 | fclose() | Disk I/O Functions *Importance*

Why is it important to call fclose() after finishing file operations?

A. To increase the speed of the CPU
B. To delete the file from the disk
C. To rename the file automatically
D. To ensure all buffered data is written to the file and system resources are freed

Ans: D

Explanation: *fclose() flushes any remaining buffered output and releases the file handle, preventing data loss or resource leaks.*

Q117 | fclose() | Disk I/O Functions *Parameters*

Which parameter does fclose() require?

A. The FILE pointer of the file to be closed

B. The filename as a string

C. The mode used to open the file

D. No parameters are required

Ans: A

Explanation: *fclose() takes the FILE pointer that identifies which open file stream should be closed.*

Q118 | fclose() | Disk I/O Functions *Scenario-based*

What could happen if a program does not call fclose() before terminating?

A. The program will fail to compile

B. Buffered data might not be written to the file, potentially causing data loss

C. The file will be automatically deleted

D. The CPU will overheat

Ans: B

Explanation: *Without fclose(), buffered data may remain unwritten, risking incomplete or lost file content.*

Q119 | fclose() | Disk I/O Functions *Applications*

What is a common application of fclose()?

A. Reading formatted data from the console

B. Declaring array variables

C. Releasing the file handle after all reading or writing operations are complete

D. Allocating heap memory for a structure

Ans: C

Explanation: *fclose() is used at the end of file processing to properly close the file and release resources.*

Q120 | fclose() | Disk I/O Functions *Common Mistakes*

Which of the following is a common mistake regarding fclose()?

A. Calling fclose() after finishing all file operations

B. Checking the return value of fclose()

C. Passing a valid FILE pointer to fclose()

D. Continuing to use a FILE pointer for read/write operations after it has already been closed

Ans: D

Explanation: *Using a FILE pointer after fclose() leads to undefined behavior since the associated resources have been released.*

Q121 | fclose() | Disk I/O Functions *Concept-based*

fclose() should typically be called:

A. Once all necessary read or write operations on the file have been completed

B. Before the file is opened with fopen()

C. Immediately after declaring the FILE pointer, before any use

D. Only if the program encounters an error

Ans: A

Explanation: *fclose() is called after all file operations are done, ensuring proper closure and resource release.*

Function Focus: fgets()

Q122 | fgets() | Disk I/O Functions *Purpose*

What is the purpose of fgets()?

A. To write a line of text to a file

B. To read a line of text (a string) from a file

C. To open a file for reading

D. To delete a line from a file

Ans: B

Explanation: *fgets() reads characters from a file into a string until a newline, EOF, or the specified limit is reached.*

Q123 | fgets() | Disk I/O Functions *Syntax*

What is the correct syntax for fgets()?

A. int fgets(char *str, FILE *stream);

B. char *fgets(FILE *stream, char *str);

C. char *fgets(char *str, int n, FILE *stream);

D. void fgets(char *str, int n);

Ans: C

Explanation: *fgets() takes a character buffer, a maximum number of characters, and a FILE pointer.*

Q124 | fgets() | Disk I/O Functions *Parameters*

What does the second parameter (n) of fgets() represent?

A. The number of lines to skip

B. The mode in which the file was opened

C. The size of the file in bytes

D. The maximum number of characters (including the null terminator) to read

Ans: D

Explanation: *The parameter n limits how many characters fgets() reads, ensuring the buffer is not overflowed.*

Q125 | fgets() | Disk I/O Functions *Return Value*

What does fgets() return when it reaches the end of the file without reading any characters?

A. NULL

B. EOF

C. 0

D. An empty integer

Ans: A

Explanation: *fgets() returns NULL if it reaches EOF before reading any characters into the buffer.*

Q126 | fgets() | Disk I/O Functions *Reading Strings*

Does fgets() include the newline character in the string it reads, if present?

A. No, the newline character is always removed automatically

B. Yes, the newline character is included in the resulting string if it fits within the buffer

C. fgets() never reads the newline character under any circumstance

D. fgets() replaces the newline with a null character before storing it

Ans: B

Explanation: *If a newline is encountered before the limit is reached, fgets() includes it in the string it stores.*

Q127 | fgets() | Disk I/O Functions *Applications*

What is a common application of fgets()?

A. Writing formatted numeric data to a file

B. Closing multiple files at once

C. Reading a text file line by line for processing

D. Deleting specific lines from a file

Ans: C

Explanation: *fgets() is commonly used in loops to read a text file one line at a time.*

Q128 | fgets() | Disk I/O Functions *File Opening Modes*

Which mode must the file be opened in for fgets() to work correctly?

A. Only "w" mode

B. Only "a" mode

C. No specific mode is required

D. A readable mode such as "r"

Ans: D

Explanation: *fgets() requires the file to have been opened for reading.*

Q129 | fgets() | Disk I/O Functions *Return Value*

What return value indicates that fgets() read the string successfully?

A. A pointer to the string (same as the buffer passed in)

B. The integer 1

C. EOF

D. The number of characters read as an integer

Ans: A

Explanation: *On success, fgets() returns the same pointer passed as its first argument, pointing to the buffer.*

Q130 | fgets() | Disk I/O Functions *Common Mistakes*

Which of the following is a common mistake when using fgets()?

A. Providing the FILE pointer as the third argument

B. Providing a buffer size (n) that is too large for the actual character array, causing a buffer overflow

C. Checking the return value for NULL

D. Using fgets() inside a while loop

Ans: B

Explanation: *The size parameter must accurately reflect the buffer's actual capacity to avoid overflow.*

Q131 | fgets() | Disk I/O Functions *Comparison-based*

fgets() is generally considered safer than which other input function because it limits the number of characters read?

A. putc()

B. fclose()

C. gets()

D. fopen()

Ans: C

Explanation: *Unlike gets(), fgets() limits input length using its size parameter, helping prevent buffer overflows.*

Function Focus: fputs()

Q132 | fputs() | Disk I/O Functions *Purpose*

What is the purpose of fputs()?

A. To read a string of characters from a file
B. To open a file for writing
C. To count characters in a file
D. To write a string of characters to a file

Ans: D

Explanation: *fputs() writes a null-terminated string to the specified output stream.*

Q133 | fputs() | Disk I/O Functions *Syntax*

What is the correct syntax for fputs()?

A. int fputs(const char *str, FILE *stream);
B. char fputs(FILE *stream, char *str);
C. void fputs(char *str);
D. int fputs(FILE *stream);

Ans: A

Explanation: *fputs() takes a string and a FILE pointer, writing the string to that stream.*

Q134 | fputs() | Disk I/O Functions *Writing Strings*

Does fputs() automatically add a newline character at the end of the string it writes?

A. Yes, it always appends a newline character
B. No, fputs() does not add a newline automatically
C. Yes, but only in append mode
D. No, but it removes any existing newline

Ans: B

Explanation: *Unlike puts(), fputs() writes exactly the string provided without adding an extra newline character.*

Q135 | fputs() | Disk I/O Functions *Return Value*

What does fputs() return on success?

A. NULL
B. The string itself
C. A non-negative value
D. The character count as a negative number

Ans: C

Explanation: *fputs() returns a non-negative value on success and EOF on failure.*

Q136 | fputs() | Disk I/O Functions *Return Value*

What does fputs() return if an error occurs during writing?

A. NULL
B. 0
C. -2
D. EOF

Ans: D

Explanation: *fputs() returns EOF to signal that an error occurred while writing the string.*

Q137 | fputs() | Disk I/O Functions *File Opening Modes*

Which mode must the file be opened in before fputs() can be used?

A. A writable mode such as "w", "a", or "r+"
B. Only "r" mode
C. No mode is required
D. Only binary read mode

Ans: A

Explanation: *The file must be opened for writing so fputs() can successfully store the string.*

Q138 | fputs() | Disk I/O Functions *Applications*

What is a common application of fputs()?

A. Reading formatted numeric input from the keyboard
B. Writing lines of text to a file for storage or logging
C. Deleting a file from disk
D. Renaming an existing file

Ans: B

Explanation: *fputs() is commonly used to write string data, such as log entries, to text files.*

Q139 | fputs() | Disk I/O Functions *Code-snippet*

Which of the following correctly writes the string "Hello" to file pointer fp using fputs()?

A. fputs(fp, "Hello");
B. fputs("Hello");
C. fputs("Hello", fp);
D. fputs(fp);

Ans: C

Explanation: *The string comes first, followed by the FILE pointer, as in fputs("Hello", fp).*

Q140 | fputs() | Disk I/O Functions *Common Mistakes*

Which of the following is a common mistake when using fputs()?

A. Passing a string as the first argument
B. Passing a FILE pointer as the second argument
C. Checking the return value for EOF
D. Assuming fputs() adds a newline automatically and omitting it when needed

Ans: D

Explanation: *Since fputs() does not add a newline, forgetting to include one manually can cause lines to run together.*

Q141 | fputs() | Disk I/O Functions *Comparison-based*

fputs() is functionally similar to which console-output function, but operates on a file stream instead?

A. puts()
B. getc()
C. fopen()
D. feof()

Ans: A

Explanation: *fputs() writes a string similarly to puts(), but targets a specified file stream rather than the console.*

Function Focus: feof()

Q142 | feof() | Disk I/O Functions *Purpose*

What is the purpose of feof()?

A. To open a file for reading
B. To check whether the end-of-file indicator has been set for a file stream
C. To close a file
D. To count the number of lines in a file

Ans: B

Explanation: *feof() tests whether the end-of-file indicator associated with a stream has been set.*

Q143 | feof() | Disk I/O Functions *Syntax*

What is the correct syntax for feof()?

A. void feof(FILE *stream);
B. int feof(char *filename);
C. int feof(FILE *stream);
D. FILE feof(int stream);

Ans: C

Explanation: *feof() takes a FILE pointer and returns a non-zero value if the end-of-file indicator is set.*

Q144 | feof() | Disk I/O Functions *Return Value*

What does feof() return if the end of the file has been reached?

A. Zero (false)
B. NULL
C. EOF as a string
D. A non-zero value (true)

Ans: D

Explanation: *feof() returns a non-zero value once the end-of-file indicator for the stream has been set.*

Q145 | feof() | Disk I/O Functions *End-of-file Detection*

When exactly does the end-of-file indicator typically get set, in relation to feof()?

A. After a read operation attempts to read past the last character in the file
B. As soon as the file is opened
C. Before the first read operation
D. Only when fclose() is called

Ans: A

Explanation: *The EOF indicator is set after a read operation fails to read further data, not simply upon opening the file.*

Q146 | feof() | Disk I/O Functions *Applications*

Which of the following is a common application of feof()?

A. Writing formatted output to the console
B. Controlling a loop that reads a file until no more data is available
C. Declaring a new FILE pointer
D. Opening a file in append mode

Ans: B

Explanation: *feof() is commonly used as a loop condition to stop reading once the end of the file is reached.*

Q147 | feof() | Disk I/O Functions *Common Mistakes*

What is a common mistake when using feof() as a loop condition immediately after fopen()?

A. Using feof() after a successful read operation

B. Passing the FILE pointer to feof()

C. Using feof() to control a loop before any read attempt can cause the loop to process one extra iteration incorrectly

D. Checking feof() only after attempting to read

Ans: C

Explanation: *Because the EOF flag is set only after a failed read, using while(!feof(fp)) before reading can process invalid data one time too many.*

Q148 | feof() | Disk I/O Functions *Concept-based*

feof() is typically used together with which type of function inside a loop?

A. A write function such as fputs() only

B. The fopen() function only

C. The fclose() function only

D. A read function such as fgets() or getc()

Ans: D

Explanation: *feof() is generally checked in combination with a read operation to determine when to stop reading.*

Q149 | feof() | Disk I/O Functions *Return Value*

What does a return value of 0 from feof() indicate?

A. The end-of-file indicator has not been set for the stream

B. The file has been successfully closed

C. The file does not exist

D. The file was opened in write mode

Ans: A

Explanation: *A return value of 0 means the end-of-file condition has not yet occurred on that stream.*

Q150 | feof() | Disk I/O Functions *Basic*

Which header must be included to use feof()?

A. stdlib.h
B. stdio.h
C. string.h
D. math.h

Ans: B

Explanation: *feof(), like other standard file functions, is declared in stdio.h.*

Q151 | feof() | Disk I/O Functions *Application-oriented*

What is the safest practice regarding feof() when reading a file?

A. Rely solely on feof() before any read attempt has been made
B. Never use feof() in file-reading programs
C. Check the return value of the read function itself, using feof() to confirm why the read failed
D. Call feof() only after fclose()

Ans: C

Explanation: *A safer approach checks whether the read function itself succeeded, using feof() to distinguish EOF from a read error.*

CHAPTER 7
Simple File-Based Programs

Q152 | File Programs | Simple File-Based Programs *Scenario-based*

In a simple C program to write to a file, which sequence of steps is correct?

A. Close the file, open the file, write data
B. Write data, open the file, close the file
C. Open the file, close the file, write data
D. Open the file, write data using fputs()/putc(), then close the file

Ans: D

Explanation: *The correct sequence is open, write, close -- ensuring proper file handling.*

Q153 | File Programs | Simple File-Based Programs *Scenario-based*

In a simple C program to read a file, which sequence of steps is correct?

A. Open the file, read data using fgets()/getc() in a loop, then close the file

B. Close the file, read data, open the file

C. Read data, close the file, open the file

D. Open the file, close the file, read data

Ans: A

Explanation: *A file must be opened before reading and closed after all reading is complete.*

Q154 | File Programs | Simple File-Based Programs *Opening Files*

What is typically checked right after calling fopen() in a simple file program?

A. Whether the loop variable is zero

B. Whether the returned FILE pointer is NULL

C. Whether the CPU is idle

D. Whether the array is sorted

Ans: B

Explanation: *Checking for a NULL FILE pointer confirms whether the file was opened successfully before proceeding.*

Q155 | File Programs | Simple File-Based Programs *Character I/O*

In a simple program that copies content from one text file to another, which pair of functions is commonly used together?

A. fopen() and fclose() only, without any read/write functions

B. feof() and fopen() only

C. getc() to read and putc() to write, character by character

D. fputs() and fopen() only

Ans: C

Explanation: *A simple file-copy program commonly reads each character with getc() and writes it with putc() in a loop.*

Q156 | File Programs | Simple File-Based Programs *Code-snippet*

Which loop condition is commonly used with getc() to read a file until its end?

A. while (getc(fp) == 0)

B. for (fp = 0; fp < 10; fp++)

C. while (fclose(fp) != NULL)

D. while ((ch = getc(fp)) != EOF)

Ans: D

Explanation: *Comparing the character read by getc() to EOF is a standard way to detect the end of the file during reading.*

Q157 | File Programs | Simple File-Based Programs *Closing Files*

What is the purpose of closing a file with fclose() at the end of a simple file-handling program?

A. To ensure all data is properly saved and system resources are released

B. To permanently delete the file from disk

C. To rename the file automatically

D. To reopen the file in a different mode

Ans: A

Explanation: *fclose() flushes any pending output and frees the resources associated with the open file.*

Q158 | File Programs | Simple File-Based Programs *End-of-file Checking*

In a simple program using fgets() to read lines from a file, what typically signals that all lines have been read?

A. fgets() returns the string "EOF"

B. fgets() returns NULL

C. fgets() returns the integer 1

D. The program automatically terminates without any check

Ans: B

Explanation: *fgets() returns NULL when there is no more data to read, signaling the end of the file has been reached.*

Q159 | File Programs | Simple File-Based Programs *Character I/O*

A simple program reading character-by-character from a file uses which structure most naturally?

A. A single if statement executed once
B. A switch statement based on file size
C. A while loop that continues until getc() returns EOF
D. A recursive function with no base case

Ans: C

Explanation: *Since the number of characters is unknown in advance, a while loop checking for EOF is the natural structure.*

Q160 | File Programs | Simple File-Based Programs *Working of FILE Structure*

What does the FILE structure's internal position member allow a simple program to do while reading character by character?

A. Manually recalculate the file's size after every read
B. Reopen the file automatically after each character
C. Skip closing the file at the end
D. Automatically advance to the next character after each successful read

Ans: D

Explanation: *The FILE structure's internal tracking allows sequential reads to automatically progress through the file.*

Q161 | File Programs | Simple File-Based Programs *String I/O*

In a simple program demonstrating string I/O, which pair of functions is most appropriate for writing and later reading a line of text?

A. fputs() to write the line and fgets() to read it back
B. putc() to write the line and fclose() to read it
C. fopen() to write the line and fopen() to read it
D. feof() to write the line and getc() to read it

Ans: A

Explanation: *fputs() writes string data, and fgets() is the corresponding function to read a line of text back from the file.*

Q162 | File Programs | Simple File-Based Programs *Working of FILE Structure*

Which of the following best demonstrates the 'working of the FILE structure' through a simple example?

A. Declaring an array of file names

B. Opening a file, observing that read operations progress sequentially through its content, then closing it

C. Printing the source code of the compiler

D. Sorting integers using bubble sort

Ans: B

Explanation: *A simple example of the FILE structure's working is seen in how sequential reads/writes track position until the file is closed.*

Q163 | File Programs | Simple File-Based Programs *Assertion-Reason*

What happens if a simple program attempts to read from a file that was opened only in "w" mode?

A. The read operation will succeed as normal

B. The file will automatically switch to read mode

C. The read operation will fail because the file was not opened for reading

D. The program will read random binary data successfully

Ans: C

Explanation: *A file opened in write-only mode ("w") does not permit read operations, so attempts to read will fail.*

Q164 | File Programs | Simple File-Based Programs *Character I/O*

In a program that counts the number of characters in a text file, which function is most directly useful?

A. fopen(), called repeatedly

B. fclose(), called repeatedly

C. putc(), called repeatedly

D. getc(), called repeatedly in a loop until EOF

Ans: D

Explanation: *Counting characters typically involves reading each character with getc() and incrementing a counter until EOF.*

Q165 | File Programs | Simple File-Based Programs *Scenario-based*

A simple program wants to append a new line to an existing log file without erasing previous entries. Which mode should be used?

A. "a"
B. "w"
C. "r"
D. "x"

Ans: A

Explanation: *The "a" mode ensures new data is added after existing content rather than overwriting the file.*

Q166 | File Programs | Simple File-Based Programs *Closing Files*

What is a likely outcome if a program forgets to call fclose() after writing data with putc()?

A. The data will always be saved instantly regardless of fclose()
B. Some buffered data may not be flushed to the file, resulting in incomplete file content
C. The program will fail to compile
D. The FILE pointer will automatically become stdin

Ans: B

Explanation: *Without fclose(), buffered output may remain unflushed, so the file might not contain all the written data.*

Q167 | File Programs | Simple File-Based Programs *Opening Files*

In a simple file-reading program, why is it good practice to check the FILE pointer returned by fopen() before proceeding?

A. To make the program compile faster
B. To automatically fix any syntax errors
C. To avoid using a NULL pointer, which would cause undefined behavior in later read operations
D. To reduce the number of lines in the code

Ans: C

Explanation: *Using a NULL FILE pointer for read/write operations leads to undefined behavior, so checking it first is essential.*

Q168 | File Programs | Simple File-Based Programs *Match-the-Following*

Which combination of functions best demonstrates a complete simple text-file program covering open, write, and close?

A. fopen(), feof(), getc()
B. fclose(), fopen(), fputs()
C. getc(), putc(), feof()
D. fopen(), fputs(), fclose()

Ans: D

Explanation: *A minimal write program follows the sequence: open the file, write to it with fputs(), then close it.*

Q169 | File Programs | Simple File-Based Programs *Match-the-Following*

Which combination of functions best demonstrates a complete simple text-file program covering open, read, and close?

A. fopen(), fgets(), fclose()
B. fopen(), fputs(), feof()
C. fclose(), fgets(), fopen()
D. putc(), fopen(), fclose()

Ans: A

Explanation: *A minimal read program follows the sequence: open the file, read from it with fgets(), then close it.*

Q170 | File Programs | Simple File-Based Programs *Tricky*

In a simple program, what does an unsuccessful feof() check right after opening (before any read) most likely indicate?

A. That the file is definitely empty
B. That the end-of-file condition has simply not been evaluated yet, since no read attempt has occurred
C. That the file failed to open
D. That the program has a syntax error

Ans: B

Explanation: *feof() only reflects the EOF status after a read attempt; checking it before any read is not meaningful.*

Q171 | File Programs | Simple File-Based Programs *Application-oriented*

What is the educational value of writing simple file-based programs using fopen(), getc()/putc(), fgets()/fputs(), feof(), and fclose() together?

A. They demonstrate how to build a compiler

B. They show how to manage CPU registers directly

C. They illustrate the complete life cycle of file handling: opening, processing data, and closing the file properly

D. They replace the need for understanding variables

Ans: C

Explanation: *Together, these functions demonstrate the full lifecycle of file handling from opening through processing to closing.*